

SmartSafe Master Unit

Complete Design and Code Explanation of the 25LC040 SPI EEPROM and Event Logger

Microprocessors End-Term Project

July 2026

Abstract

This report explains the external nonvolatile memory subsystem used by the SmartSafe Master unit. The design uses the ATmega32 hardware SPI peripheral to communicate with a Microchip 25LC040A serial EEPROM. The report covers the electrical connections, SPI register configuration, clock-rate choice, byte-transfer mechanism, chip-select framing, instruction set, write-enable latch, write-in-progress polling, full-duplex dummy transfers, and the unusual encoding of the ninth address bit inside bit 3 of the READ or WRITE command. The second half documents the event-logging layer: memory map, persistent header, 8-byte records, big-endian timestamp storage, optional event data, XOR checksum, circular-buffer behavior, initialization validation, log clearing, and USART log dumping.

Contents

1	Role of the External EEPROM	3
2	25LC040A Component Overview	3
2.1	Device pins	3
3	Why Hardware SPI Is Used	4
4	SPI Register Configuration	4
4.1	Data-direction register	5
4.2	SPCR fields	5
4.3	SPSR and SPI clock	5
5	Chip-Select Framing	6
6	Full-Duplex Byte Transfer	6
7	25LC040A Instruction Set	7
8	The Ninth Address Bit	7
8.1	Why a ninth bit is necessary	7
8.2	Code implementation	7
9	Write-Enable Latch	8
10	Writing One Byte	8
11	Status Register and Ready Polling	9

12 Reading One Byte	10
13 Memory Map Used by the Logger	10
14 Header Format	11
15 Logger Initialization and Clearing	11
16 Record Format	11
17 Checksum	12
18 Writing an Event Record	12
19 Circular-Buffer Ordering	13
20 Log Dump Through USART	13
21 Timing, Performance, and Endurance Considerations	14
22 Common Hardware and Firmware Faults	14
23 Proteus Verification Procedure	14
24 Complete SPI EEPROM Driver Listing	15
25 Conclusion	16

1 Role of the External EEPROM

The internal ATmega32 EEPROM stores configuration that is small and directly associated with the controller, such as the four-digit password and failed-attempt count. The external 25LC040A is used for the event history because it provides a separate 512-byte nonvolatile memory accessed through SPI. The log survives reset and simulated power events and can be dumped later through USART.

The external EEPROM subsystem is divided into two layers:

1. **SPI EEPROM driver:** transfers command, address, status, and data bytes between the ATmega32 and the 25LC040A.
2. **Logger:** converts application events into structured records, maintains circular-buffer metadata, verifies checksums, and prints records to the terminal.

2 25LC040A Component Overview

The 25LC040A is a 4-Kbit serial EEPROM organized as 512 bytes by 8 bits:

$$4096 \text{ bits} = 512 \text{ bytes}, \quad 0 \leq \text{address} \leq 511 = 0x1FF.$$

Because 512 locations require nine address bits, the address is $A_8A_7 \dots A_0$. Unlike larger SPI EEPROMs that send a complete high address byte, the 25LC040A embeds A_8 in the instruction byte and sends only $A_7 \dots A_0$ as the following address byte.

2.1 Device pins

Table 1: 25LC040A pins and SmartSafe connections.

Pin	Signal	SmartSafe connection and function
1	CS	Connected to ATmega32 PB4/SS. Active low; frames every SPI command.
2	SO	Connected to PB6/MISO. EEPROM serial-data output.
3	WP	Tied to +5 V. Prevents unintended hardware write protection.
4	VSS	Ground.
5	SI	Connected to PB5/MOSI. EEPROM serial-data input.
6	SCK	Connected to PB7/SCK. Clock generated by the Master.
7	HOLD	Tied to +5 V. Disables the asynchronous bus-pause function.
8	VCC	+5 V, with a local 100 nF decoupling capacitor to ground.

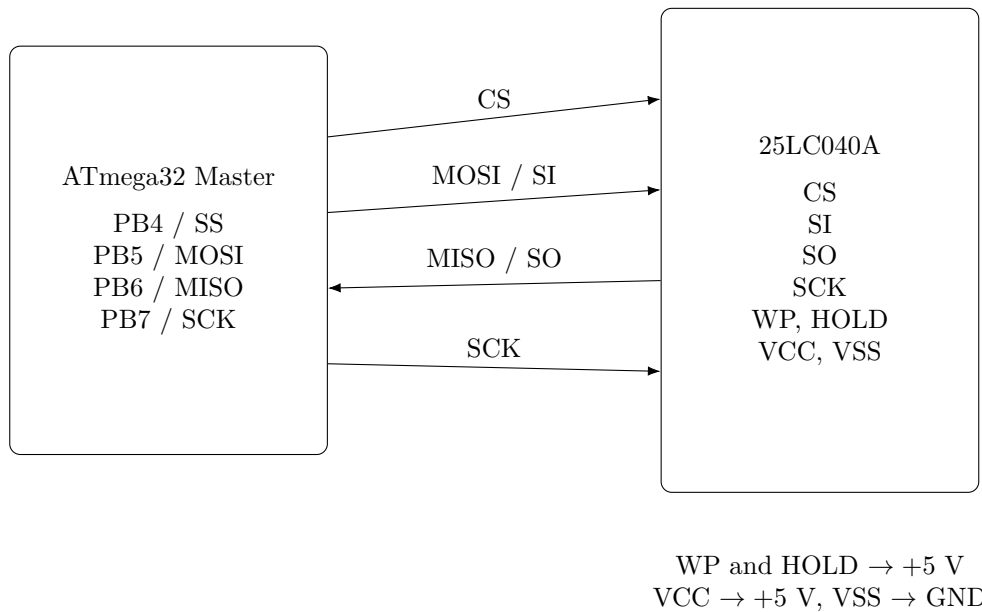


Figure 1: Hardware SPI connection between the Master and 25LC040A.

3 Why Hardware SPI Is Used

The latest Master implementation uses the ATmega32 hardware SPI peripheral rather than bit-banging the clock and data pins. Hardware SPI provides:

- precise clock generation;
- automatic serial shifting through SPDR;
- a completion flag, SPIF;
- lower CPU overhead and simpler code;
- standard use of the dedicated PB4–PB7 SPI pins.

The Master is the only device that generates the clock. The EEPROM is an SPI slave and responds only while CS is low.

4 SPI Register Configuration

```

1 void spi_eeprom_init(void)
2 {
3     EE_SPI_DDR |= (1 << EE_SPI_SS) |
4                 (1 << EE_SPI_MOSI) |
5                 (1 << EE_SPI_SCK);
6     EE_SPI_DDR &= ~(1 << EE_SPI_MISO);
7     ee_cs_high();
8
9     /* Enable SPI, Master, mode 0, clock F_CPU/16 = 500 kHz @ 8 MHz. */
10    SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR0);
11    SPSR = 0x00;
12 }

```

Listing 1: SPI EEPROM initialization.

4.1 Data-direction register

The Master drives SS, MOSI, and SCK, so they are outputs. The EEPROM drives MISO, so it is an input:

```
1 DDRB4 = 1; /* CS/SS output */
2 DDRB5 = 1; /* MOSI output */
3 DDRB6 = 0; /* MISO input */
4 DDRB7 = 1; /* SCK output */
```

Keeping the hardware SS pin configured as an output is also important for preserving SPI Master mode on classic AVR devices.

4.2 SPCR fields

The configured value is

```
1 SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPR0);
```

Table 2: SPI Control Register choices.

Bit	Field	Selected behavior
7	SPIE	Zero: SPI interrupt disabled; transfers are polled.
6	SPE	One: SPI peripheral enabled.
5	DORD	Zero: most-significant bit is transmitted first.
4	MSTR	One: ATmega32 operates as SPI Master.
3	CPOL	Zero: SCK is low while idle.
2	CPHA	Zero: data is sampled on the leading rising edge; SPI mode 0.
1:0	SPR1:SPR0	01: base clock division of 16 when SPI2X=0.

The 25LC040A supports SPI mode 0, so CPOL=0 and CPHA=0 are appropriate. Mode 0 behavior is:

- SCK is low when idle;
- output data changes around the falling edge;
- input data is sampled on the rising edge.

4.3 SPSR and SPI clock

The code writes

```
1 SPSR = 0x00;
```

so SPI2X=0. With SPR1:SPR0=01,

$$f_{SCK} = \frac{F_{CPU}}{16} = \frac{8 \text{ MHz}}{16} = 500 \text{ kHz}.$$

A 500 kHz bus is far below the EEPROM maximum and provides generous timing margin in Proteus and real hardware. It is also fast enough that the serial transfer time is negligible compared with the EEPROM's internal write cycle.

5 Chip-Select Framing

```

1 static void ee_cs_low(void)
2 {
3     EE_SPI_PORT &= ~(1 << EE_SPI_SS);
4 }
5
6 static void ee_cs_high(void)
7 {
8     EE_SPI_PORT |= (1 << EE_SPI_SS);
9 }

```

CS is active low. A command begins when the Master drives CS low and ends when it returns CS high. The EEPROM interprets bytes according to their position within that interval. CS is set high before enabling SPI so that the EEPROM is not accidentally selected during initialization.

For writes, the low-to-high transition on CS after the command, address, and data have been sent causes the EEPROM to begin its internal self-timed write cycle.

6 Full-Duplex Byte Transfer

```

1 static uint8_t spi_transfer(uint8_t data)
2 {
3     SPDR = data;
4     while (!(SPSR & (1 << SPIF))) {
5         /* wait */
6     }
7     return SPDR;
8 }

```

Listing 2: Hardware SPI byte transfer.

SPI is full duplex. For every bit shifted from MOSI, one bit is simultaneously shifted into MISO. Writing a byte to SPDR starts the hardware transfer. The processor polls SPIF until eight clock pulses have completed, then reads the received byte from SPDR.

On the ATmega32, the normal flag-clearing sequence is to read SPSR while SPIF is set and then access SPDR. The polling condition reads SPSR; the return statement reads SPDR, completing that sequence.

At 500 kHz, one bit requires 2 μ s and one byte requires approximately

$$8 \times 2 \mu\text{s} = 16 \mu\text{s}.$$

Transaction overhead is therefore much shorter than the millisecond-scale internal EEPROM write operation.

7 25LC040A Instruction Set

Table 3: Important 25LC040A instructions.

Hex	Mnemonic	Function
0x06	WREN	Set the write-enable latch before a WRITE or status-register write.
0x04	WRDI	Clear the write-enable latch. Supported by the device but not needed by the current driver.
0x05	RDSR	Read the status register, including WIP and WEL.
0x01	WRSR	Write status-register protection bits. Not used by the project.
0x03/0x0B	READ	Read from lower/upper half of memory; bit 3 carries address bit A_8 .
0x02/0x0A	WRITE	Write to lower/upper half of memory; bit 3 carries address bit A_8 .

The project defines the base opcodes:

```

1 #define EE25_WREN    0x06
2 #define EE25_WRITE   0x02
3 #define EE25_READ    0x03
4 #define EE25_RDSR    0x05

```

The upper-memory versions are generated by OR-ing bit 3, 0x08, into READ or WRITE.

8 The Ninth Address Bit

8.1 Why a ninth bit is necessary

The memory has 512 byte locations:

$$512 = 2^9.$$

Thus the complete address is

$$A_8 A_7 A_6 A_5 A_4 A_3 A_2 A_1 A_0.$$

Only one address byte follows the instruction, so it can carry $A_7 \dots A_0$. The remaining bit, A_8 , is encoded in bit 3 of the READ or WRITE instruction.

8.2 Code implementation

```

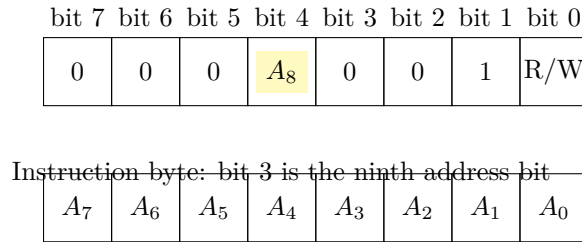
1 spi_transfer((uint8_t)(EE25_WRITE |
2                 ((addr & 0x0100) ? 0x08 : 0x00)));
3 spi_transfer((uint8_t)(addr & 0xFF));

```

The mask 0x0100 tests A_8 . If it is one, 0x08 is inserted into the command. The following byte contains the lower eight address bits.

Table 4: Address examples.

Address	A_8	Low byte	READ command	WRITE command	Interpretation
0x07A	0	0x7A	0x03	0x02	Lower half
0x0FF	0	0xFF	0x03	0x02	Last lower-half byte
0x100	1	0x00	0x0B	0x0A	First upper-half byte
0x17A	1	0x7A	0x0B	0x0A	Upper half
0x1FF	1	0xFF	0x0B	0x0A	Final device byte



Following address byte

Figure 2: Encoding of the 9-bit address across the command and address bytes. The low opcode bits differ between READ and WRITE.

Before every operation, the driver limits the supplied address to the physical range:

```
1 addr &= 0x01FF;
```

This keeps only bits $A_8 \dots A_0$ and maps any larger value into the 512-byte address space.

9 Write-Enable Latch

EEPROM writes are protected by a write-enable latch, WEL. The Master must issue WREN as a separate CS-framed transaction before each write:

```
1 static void ext_eeprom_write_enable(void)
2 {
3     ee_cs_low();
4     spi_transfer(E25_WREN);
5     ee_cs_high();
6 }
```

The separate CS rising edge completes the WREN command and sets WEL. The following WRITE command is then accepted. After the internal write cycle begins or completes, the device automatically clears WEL, so the driver repeats WREN for every byte write.

10 Writing One Byte

```
1 void ext_eeprom_write_byte(uint16_t addr, uint8_t data)
2 {
3     addr &= 0x01FF;
4     ext_eeprom_write_enable();
5
6     ee_cs_low();
```

```

7  spi_transfer((uint8_t)(EE25_WRITE |
8      ((addr & 0x0100) ? 0x08 : 0x00)));
9  spi_transfer((uint8_t)(addr & 0xFF));
10 spi_transfer(data);
11 ee_cs_high();
12
13 ext_eeprom_wait_ready();
14 }

```

Listing 3: Single-byte write sequence.

The complete sequence is:

1. Send WREN with its own CS-low interval.
2. Drive CS low again.
3. Send WRITE with A_8 embedded in bit 3.
4. Send $A_7 \dots A_0$.
5. Send the data byte.
6. Drive CS high, starting the self-timed erase/write cycle.
7. Poll the WIP status bit until it becomes zero.

11 Status Register and Ready Polling

```

1  static void ext_eeprom_wait_ready(void)
2  {
3      uint8_t sr;
4      do {
5          ee_cs_low();
6          spi_transfer(EE25_RDSR);
7          sr = spi_transfer(0xFF);
8          ee_cs_high();
9      } while (sr & 0x01);
10 }

```

Listing 4: Waiting for the internal write cycle.

The important status bits are:

Bit	Name	Meaning
0	WIP	Write in progress. One means the internal write cycle is active.
1	WEL	Write-enable latch. One means a write command is currently permitted.
3:2	BP1:BP0	Block-protection selection. Left at the device default in this project.
7	WPEN	Status-register hardware write-protect enable. Not configured by this driver.

After sending RDSR, the Master sends 0xFF. That byte is not meaningful command data; it supplies eight SCK pulses so the EEPROM can shift its status byte onto MISO. Polling rather

than using a fixed delay finishes as soon as the device is ready and remains correct across operating conditions.

12 Reading One Byte

```

1 uint8_t ext_eeprom_read_byte(uint16_t addr)
2 {
3     uint8_t data;
4     addr &= 0x01FF;
5
6     ee_cs_low();
7     spi_transfer((uint8_t)(EE25_READ |
8                         ((addr & 0x0100) ? 0x08 : 0x00)));
9     spi_transfer((uint8_t)(addr & 0xFF));
10    data = spi_transfer(0xFF);
11    ee_cs_high();
12
13    return data;
14 }
```

Listing 5: Single-byte read sequence.

The dummy 0xFF transfer is required because SPI has no separate “read” clock operation. The Master must transmit something to generate the eight clock pulses that shift the selected EEPROM byte onto MISO. The value transmitted during this phase is irrelevant; 0xFF is conventional.

13 Memory Map Used by the Logger

The logger reserves the first eight bytes as a persistent header and uses the remaining 504 bytes for 63 records:

Table 5: SmartSafe external EEPROM map.

Address range	Size	Purpose
0x000–0x007	8 bytes	Header: signature, next-write index, valid-record count, reserved bytes
0x008–0x1FF	504 bytes	63 records × 8 bytes

The constants are:

```

1 #define LOG_HEADER_ADDR 0x000
2 #define LOG_BASE_ADDR 0x008
3 #define LOG_RECORD_SIZE 8
4 #define LOG_MAX_RECORDS 63
5 #define LOG_MAGIC0 0x53
6 #define LOG_MAGIC1 0x46
```

The two magic bytes are ASCII ‘S’ and ‘F’. They distinguish a valid initialized SmartSafe log from blank or unrelated EEPROM contents.

14 Header Format

Table 6: Persistent 8-byte header.

Offset	Value	Meaning
0	0x53	Magic byte 0, ASCII S.
1	0x46	Magic byte 1, ASCII F.
2	log_head	Index of the next record slot to be written.
3	log_count	Number of valid records, from 0 through 63.
4–7	0xFF	Reserved for future metadata.

log_head and log_count are also kept in RAM for fast access. The header is rewritten after every new record so the circular-buffer position survives reset.

15 Logger Initialization and Clearing

```

1 void logger_init(void)
2 {
3     uint8_t m0 = ext_eeprom_read_byte(LOG_HEADER_ADDR + 0);
4     uint8_t m1 = ext_eeprom_read_byte(LOG_HEADER_ADDR + 1);
5
6     if (m0 != LOG_MAGIC0 || m1 != LOG_MAGIC1) {
7         log_clear();
8         return;
9     }
10
11     log_head = ext_eeprom_read_byte(LOG_HEADER_ADDR + 2);
12     log_count = ext_eeprom_read_byte(LOG_HEADER_ADDR + 3);
13
14     if (log_head >= LOG_MAX_RECORDS || log_count > LOG_MAX_RECORDS)
15         log_clear();
16 }
```

Listing 6: Persistent-header validation.

On startup, the logger first checks the signature. Blank EEPROM generally reads as 0xFF, so the signature fails and log_clear() creates a valid empty header. Even when the signature is correct, range checks prevent corrupted metadata from producing out-of-range addresses.

```

1 void log_clear(void)
2 {
3     log_head = 0;
4     log_count = 0;
5     write_header();
6 }
```

The implementation clears the logical log by resetting metadata. It does not erase every record byte. Old data remains physically present but is no longer included because log_count=0.

16 Record Format

Each event occupies eight bytes:

Table 7: Event-record structure.

Byte	Field	Description
0	Event code	One of the <code>EVT_*</code> constants.
1	Timestamp[31:24]	Most-significant timestamp byte.
2	Timestamp[23:16]	Timestamp byte.
3	Timestamp[15:8]	Timestamp byte.
4	Timestamp[7:0]	Least-significant timestamp byte.
5	Data[15:8]	Optional event-specific data, high byte.
6	Data[7:0]	Optional event-specific data, low byte.
7	Checksum	XOR of bytes 0 through 6.

The timestamp is stored big-endian, so the most-significant byte appears first. This produces a consistent byte order independent of the AVR's native little-endian representation.

17 Checksum

```

1 static uint8_t checksum7(const uint8_t *b)
2 {
3     uint8_t x = 0;
4     for (uint8_t i = 0; i < 7; i++)
5         x ^= b[i];
6     return x;
7 }

```

The checksum is an XOR of the first seven bytes:

$$C = b_0 \oplus b_1 \oplus \cdots \oplus b_6.$$

It detects many single-bit and simple corruption patterns and is inexpensive on an 8-bit MCU. It is not a cryptographic integrity mechanism and is weaker than a CRC, but it is sufficient for identifying obviously invalid log records in this educational design.

18 Writing an Event Record

```

1 void log_event_data(uint8_t event_code,
2                     uint32_t timestamp,
3                     uint16_t data)
4 {
5     uint8_t r[LOG_RECORD_SIZE];
6     uint16_t base;
7
8     if (log_head >= LOG_MAX_RECORDS)
9         log_head = 0;
10
11     r[0] = event_code;
12     r[1] = (uint8_t)(timestamp >> 24);
13     r[2] = (uint8_t)(timestamp >> 16);
14     r[3] = (uint8_t)(timestamp >> 8);
15     r[4] = (uint8_t)(timestamp);
16     r[5] = (uint8_t)(data >> 8);
17     r[6] = (uint8_t)(data);
18     r[7] = checksum7(r);
19 }

```

```

20     base = LOG_BASE_ADDR +
21         (uint16_t)log_head * LOG_RECORD_SIZE;
22
23     for (uint8_t i = 0; i < LOG_RECORD_SIZE; i++)
24         ext_eeprom_write_byte(base + i, r[i]);
25
26     log_head++;
27     if (log_head >= LOG_MAX_RECORDS)
28         log_head = 0;
29
30     if (log_count < LOG_MAX_RECORDS)
31         log_count++;
32
33     write_header();
34 }

```

Listing 7: Record construction and circular-buffer update.

The base address is

$$base = 0x008 + 8 \times log_head.$$

After writing, `log_head` advances and wraps from 62 back to 0. `log_count` increases until it reaches 63, then remains saturated. At that point every new event overwrites the oldest record.

The convenience function

```

1 void log_event(uint8_t event_code, uint32_t timestamp)
2 {
3     log_event_data(event_code, timestamp, 0xFFFF);
4 }

```

uses 0xFFFF when an event has no additional 16-bit data.

19 Circular-Buffer Ordering

When fewer than 63 records exist, valid records begin at index zero. When the log is full, `log_head` points to the oldest record because it is the slot that will be overwritten next. The dump therefore computes:

```

1 uint8_t start = (log_count == LOG_MAX_RECORDS) ? log_head : 0;

```

It then walks `log_count` records using modulo arithmetic:

```

1 uint8_t idx = (uint8_t)((start + n) % LOG_MAX_RECORDS);

```

This prints records from oldest to newest even after several wraparounds.

20 Log Dump Through USART

The dump begins and ends with explicit markers:

```

1 usart_print("=== LOG DUMP ===\r\n");
2 ...
3 usart_print("=== END ===\r\n");

```

The Slave uses these markers to ignore log lines while the Virtual Terminal displays them.

For each record, the logger:

1. resets the watchdog so a long dump does not reset the Master;
2. reads all eight bytes;
3. verifies the checksum;
4. reconstructs the 32-bit timestamp and 16-bit data field;
5. prints the index, hexadecimal event code, event name, timestamp, and data.

A typical line is:

```
IDX=12 EVT=0x05 POWER_FAIL TS=73s DATA=0xFFFF
```

A failed checksum is reported as:

```
IDX=12 BAD_CHECKSUM
```

21 Timing, Performance, and Endurance Considerations

The current logger intentionally uses the simple byte-write API. One event writes eight record bytes and then eight header bytes. Every byte write requires WREN, a WRITE transaction, and a self-timed write cycle. This is easy to understand and robust for a small academic project, but it is not the most efficient use of the EEPROM.

The device supports page writes, with a page size of 16 bytes. A future optimization could write the complete 8-byte record in one page operation and reduce the header update to fewer cycles. This would shorten event logging and reduce write-cycle consumption. Care must be taken not to cross a page boundary in a single page-write command because data can wrap within the page.

The header is rewritten after every event, so its cells receive more writes than record cells. For a production design, rotating or journaling metadata would improve wear distribution. For the expected event rate of the SmartSafe demonstration, the simple approach is adequate.

22 Common Hardware and Firmware Faults

Symptom	Likely cause
Driver waits forever in <code>ext_eeprom_wait_ready()</code>	EEPROM unpowered; SO/MISO floating high; SI and SO swapped; HOLD low; incorrect CS wiring.
Reads always return 0xFF	EEPROM not selected, unpowered, or MISO disconnected; blank memory is also 0xFF, so verify SPI waveforms.
Writes do not persist	WREN missing; WP/HOLD wiring incorrect; CS not raised between WREN and WRITE; write cycle not completed.
Lower half works but upper half fails	Ninth address bit not inserted into command bit 3.
SPI exits Master mode unexpectedly	Hardware SS pin configured as an input and driven low; keep PB4 as output.
Corrupted records after reset	Header or record write interrupted; checksum identifies incomplete records, but metadata itself currently has no checksum.
Terminal dump activates Slave alarm	Slave uses substring matching rather than exact event messages or does not ignore the dump markers.

23 Proteus Verification Procedure

A systematic test is:

1. Confirm CS is high while idle.
2. Trigger an event and observe a WREN command: CS low, opcode 0x06, CS high.
3. Observe the WRITE command with command, low address byte, and data.
4. Observe repeated RDSR transactions until WIP clears.
5. Generate enough events to cross address 0xFF; confirm that upper-half commands become 0x0A or 0x0B.
6. Reset the Master and verify that the header restores `log_head` and `log_count`.
7. Send the terminal dump command and verify checksum, timestamp, and circular ordering.

Useful logic-analyzer channels are PB4/CS, PB5/MOSI, PB6/MISO, and PB7/SCK.

24 Complete SPI EEPROM Driver Listing

```

1  /* spi_eeprom.c - Hardware SPI driver for 25LC040 */
2
3  #include <avr/io.h>
4  #include "config.h"
5  #include "spi_eeprom.h"
6
7  static void ee_cs_low(void)
8  {
9      EE_SPI_PORT &= ~(1 << EE_SPI_SS);
10 }
11
12 static void ee_cs_high(void)
13 {
14     EE_SPI_PORT |= (1 << EE_SPI_SS);
15 }
16
17 static uint8_t spi_transfer(uint8_t data)
18 {
19     SPDR = data;
20     while (!(SPSR & (1 << SPIF))) {
21         /* wait */
22     }
23     return SPDR;
24 }
25
26 void spi_eeprom_init(void)
27 {
28     EE_SPI_DDR |= (1 << EE_SPI_SS) |
29                 (1 << EE_SPI_MOSI) |
30                 (1 << EE_SPI_SCK);
31     EE_SPI_DDR &= ~(1 << EE_SPI_MISO);
32     ee_cs_high();
33
34     SPCR = (1 << SPE) | (1 << MSTR) | (1 << SPRO);
35     SPSR = 0x00;
36 }
37
38 static void ext_eeprom_write_enable(void)
39 {
40     ee_cs_low();
41     spi_transfer(EE25_WREN);

```

```

42     ee_cs_high();
43 }
44
45 static void ext_eeprom_wait_ready(void)
46 {
47     uint8_t sr;
48     do {
49         ee_cs_low();
50         spi_transfer(EE25_RDSR);
51         sr = spi_transfer(0xFF);
52         ee_cs_high();
53     } while (sr & 0x01);
54 }
55
56 void ext_eeprom_write_byte(uint16_t addr, uint8_t data)
57 {
58     addr &= 0x01FF;
59     ext_eeprom_write_enable();
60
61     ee_cs_low();
62     spi_transfer((uint8_t)(EE25_WRITE |
63         ((addr & 0x0100) ? 0x08 : 0x00)));
64     spi_transfer((uint8_t)(addr & 0xFF));
65     spi_transfer(data);
66     ee_cs_high();
67
68     ext_eeprom_wait_ready();
69 }
70
71 uint8_t ext_eeprom_read_byte(uint16_t addr)
72 {
73     uint8_t data;
74     addr &= 0x01FF;
75
76     ee_cs_low();
77     spi_transfer((uint8_t)(EE25_READ |
78         ((addr & 0x0100) ? 0x08 : 0x00)));
79     spi_transfer((uint8_t)(addr & 0xFF));
80     data = spi_transfer(0xFF);
81     ee_cs_high();
82
83     return data;
84 }

```

Listing 8: Complete hardware-SPI EEPROM driver.

25 Conclusion

The external EEPROM subsystem combines a conventional ATmega32 hardware-SPI driver with a compact persistent circular log. The most device-specific detail is the ninth address bit: A_8 is transmitted in instruction bit 3, producing READ opcodes 0x03/0x0B and WRITE opcodes 0x02/0x0A. Reliable writes also depend on the separate WREN transaction and WIP polling. Above this driver, the logger uses a validated header, 63 fixed-size records, big-endian timestamps, optional data, and an XOR checksum to preserve and present the SmartSafe event history.

References

- [1] Microchip Technology, *25AA040A/25LC040A 4-Kbit SPI Serial EEPROM Data Sheet*.
<https://ww1.microchip.com/downloads/en/DeviceDoc/21827H.pdf>
- [2] Microchip Technology, *ATmega32/L Datasheet*, SPI Serial Peripheral Interface section.
<https://ww1.microchip.com/downloads/en/DeviceDoc/doc2503.pdf>
- [3] SmartSafe Microprocessors End-Term Project Specification, Spring 1404.